

## **CHRONIO**

> Aurora Loprino - [aurora.loprino@studio.unibo.it](mailto:aurora.loprino@studio.unibo.it)

> Enrico Poli - [enrico.poli@studio.unibo.it](mailto:enrico.poli@studio.unibo.it)

Link Repo: <https://github.com/auroraloprino/PSM-Chronio>

.....

## **SCOPO DEL PROGETTO**

Chronio è un'applicazione per la gestione del tempo personale e delle attività, che raccoglie in un unico spazio sia gli impegni pianificati che le attività da portare a termine.

L'utente può tenere traccia dei propri impegni tramite un calendario con promemoria e organizzare le proprie attività in bacheche kanban (tipo Trello), suddivise in colonne e liste personalizzabili.

.....

## **Funzionalità Principali**

### **Calendario - Aurora Loprino**

Per quanto riguarda il calendario, le funzionalità principali previste sono:

- la creazione, modifica ed eliminazione di eventi con titolo, descrizione, orario di inizio/fine (o durata "tutto il giorno") e notifiche configurabili x minuti prima tramite WorkMANager.
- tre view del calendario: mensile, settimanale e giornaliera.
- tag colorati associabili ad eventi, con la possibilità di filtrare per tag.

- persistenza locale dei dati tramite database Room. Come funzione non principale, abbiamo pensato di supportare il tema chiaro/scuro con preferenza salvata.

Per quanto riguarda le API abbiamo:

- previsioni meteo giornaliere (display mese per mese) ottenute tramite OpenMeteo con rilevamento automatico della posizione GPS.
  - festività nazionali italiane ottenute con Nager.Date, visualizzate nel calendario con un indicatore dedicato (pallino rosso).
- 

## **Bacheche - Enrico Poli**

Per quanto riguarda le bacheche, le funzionalità principali previste sono:

- la creazione, modifica ed eliminazione di bacheche, ciascuna con titolo, descrizione opzionale e un'immagine di copertina.
- all'interno di ogni bacheca, colonne personalizzabili (creazione, rinomina ed eliminazione), riordinabili tramite drag&drop orizzontale.
- card con titolo, descrizione, tag colorati e stato di completamento (fatto/da fare), spostabili tra colonne o riordinabili all'interno della stessa colonna tramite drag&drop, con anteprima in tempo reale del punto di inserimento.
- tag condivisi tra tutte le bacheche, filtrabili con modalità OR (almeno un tag) o AND (tutti i tag selezionati).
- persistenza locale tramite lo stesso database Room usato dal calendario

Per quanto riguarda le API, è stata integrata la ricerca di immagini di copertina tramite Wikimedia Commons, che restituisce foto libere da diritti con relativo autore, usate come copertina delle bacheche.

.....

## STRUTTURA

### Calendario - Aurora Loprino

Il progetto è composto da quattro moduli, con separazione delle responsabilità.

- **app/** si occupa dell'avvio dell'applicazione e del sistema delle notifiche.
- **domain/** si occupa dei modelli dati, delle interfacce repository e dello use case.
- **data/** si occupa del database Room, delle chiamate API e delle implementazioni repository.
- **ui/** si occupa delle schermate, del ViewModel e delle componenti Compose.

---

### DOMAIN/

Il modulo domain contiene la logica di business, senza dipendenze.

**EventModel** ha gli eventi, classificati da id, titolo, descrizione, start/endTime, allDay, reminderMinutes e tagIds.

**TagModel** crea i tag con id, nome, colore (in hex) ed un flag "isSystem" se sono tag "di sistema" (festività).

**WeatherModel** fornisce la previsione meteo ogni giorno, con tempMax/Min.

**HolidayModel** indica infine le festività con data e nome.

**Use case** incapsula ogni operazione di business e *invoke()* permette di chiamarle come funzioni. **UseCasesProvider** viene inizializzato solo una volta all'avvio, e tutte le parti dell'app accedono agli use case tramite questo provider.

...

## DATA/

Data implementa le interfacce definite in domain.

Il **database Room** ha tabelle per events, tags e relazioni molti a molti tra eventi e tag.

**Data Access Object** come **EventDao** e **TagDao** espongono le query come funzioni e Room genera il codice.

EventDao usa **@Insert** con **OnConflictStrategy.REPLACE**, che sovrascrive in caso di conflitto), **@Update** e **@Delete**.

Le query che restituiscono EventWithTags sono annotate con **@Transaction**.

Room esegue due query separate (una sulla tabella events e una JOIN su **event\_tag\_cross\_ref** e **tags**, per recuperare tutti i tag associati a un evento specifico tramite il suo eventId.)

Le **Repository Implementations** convertono tra i modelli del domain e le entry del DB.

Per esempio, **TagRepositoryImpl** implementa **TagRepository**.

**update()** chiama **tagDao.insert()** e sovrascrive automaticamente se esiste già un tag con lo stesso id.

**WeatherRepositoryImpl** implementa **WeatherRepository** e usa **Retrofit** per chiamare **OpenMeteo** mentre **HolidayRepositoryImpl** implementa **HolidayRepository** e chiama **Nager.Date**.

**API remote:**

**WeatherApi** chiama <https://api.open-meteo.com/v1/forecast>, passando le coordinate GPS rilevate automaticamente e un intervallo di date, per ottenere temperatura massima/minima e weather code giorno per giorno. La risposta viene mappata in **WeatherModel** e mostrata mese per mese nella vista calendario.

**HolidayApi** chiama <https://date.nager.at/api/v3/PublicHolidays/{year}/IT> per ottenere la lista delle festività nazionali italiane dell'anno corrente, mostrate nel calendario con un indicatore dedicato.

...

## APP/

**CustomApplication** inizializza **UseCasesProvider** e **ReminderSchedulerImpl** all'avvio, creando il canale per le notifiche. **NotificationScheduler** schedula le notifiche tramite **WorkManager**, ritardandole quanto necessario.

**ReminderWorker** è eseguito da **WorkManager** quando è il momento e mostra la notifica tramite **NotificationManager**.

...

## UI/

**CalendarViewModel** è la parte principale della UI.

Esponi i metodi per la navigazione, i filtri, il CRUD per eventi e tag, carica il meteo e le festività. Tutte le operazioni usano **viewModelScope.launch** con coroutine se asincrone.

**CalendarScreen** orchestra tutti i componenti figli, gestisce location e notifiche tramite **Accompanist**.

**MonthGrid** mostra la griglia mensile, **WeekView** e **DayView** mostrano griglia settimanale/giornaliera con orari “Giorno/Giornata” - 00:00/23:00.

**EventListView** mostra gli eventi sotto la vista mensile con “Oggi” e “Questa Settimana”.

**EventForm** permette le operazioni di CRUD con un data/time picker, selezione tag e validazione. **TagManagerSheet** e **TagFilterRow** gestiscono e filtrano i tag. Per comodità ci sono solo 12 colori predefiniti, in futuro sarebbe interessante mettere un color picker.

**CalendarUtils** raccoglie le funzioni utili per manipolare il timestamp in millisecondi.

- Normalizza **start/endOfDay**, **start/endOfMonth** e **startOfWeek**, importando ore, minuti, secondi e millisecondi a zero, mantenendo anno, mese e giorno.
  - Sposta un timestamp in avanti o indietro nel tempo usando **Calendar.add()**, che gestisce il cambio mese o l'anno bisestile.
  - Confronta **isSameDay**, **isSameWeek**, **eventSpansDay** (lui verifica se un evento copre almeno un istante del giorno dato.)
  - Formatta convertendo timestamp in stringhe tramite **SimpleDateFormat**, usate nei label dell'interfaccia.
- 

**Bacheche - Enrico Poli**

**DOMAIN/**

**BoardModel**, **ColumnModel**, **CardModel**, **BoardTagModel**, **PhotoModel**:  
modelli di bacheca, colonna, card, tag e foto di copertina.

Use case dedicati per ogni operazione (CRUD, spostamento/riordino, ricerca foto), esposti tramite lo stesso **UseCasesProvider** del calendario.

...

## DATA/

**ChronioDatabase** (v4, migrazione da 3 a 4) aggiunge le tabelle **boards**, **board\_columns**, **cards**, **board\_tags** e **card\_tag\_cross\_ref**.

Foreign key con **onDelete = CASCADE** tra bacheca, colonna, card e card/tag: eliminare una bacheca elimina automaticamente tutto ciò che contiene.

**BoardDao**, **ColumnDao**, **CardDao**, **BoardTagDao** espongono le query CRUD, più **nextPosition()** e **moveToColumn()** per gestire l'ordinamento.

Le Repository Implementations convertono tra entity Room e modelli domain, come per il calendario.

API remote: **WikimediaApi** cerca immagini su Wikimedia Commons

(<https://commons.wikimedia.org/>) per le copertine delle bacheche;

**PhotoRepositoryImpl** mappa la risposta e distingue errori di rete da errori del server.

...

## APP/

Nessun componente dedicato (notifiche o servizi in background): le bacheche usano però il permesso **android.permission.INTERNET** dichiarato nel manifest di app/, necessario per la ricerca delle immagini su Wikimedia Commons.

...

## UI/

**BoardListViewModel/BoardListScreen**: lista delle bacheche, con creazione, modifica ed eliminazione.

**BoardViewModel**: combina i flow di colonne, card e tag, applica i filtri OR/AND e gestisce lo stato temporaneo del drag&drop.

**BoardScreen:** vista a colonne (LazyRow) con drag&drop di colonne e card tramite la libreria reorderable, overlay che segue il dito e scroll automatico ai bordi dello schermo.

**ColumnItem, CardItem, TagFilterRow:** componenti di colonna, card e filtro per tag.

**BoardDialog, ColumnDialog, CardDialog, TagDialog:** form di creazione/modifica. **BoardDialog** integra **PhotoPickerSheet** per scegliere la copertina, con ricerca su Wikimedia Commons e debounce sulla digitazione.

.....

## PUNTI DI FORZA

### Calendario - Aurora Loprino

Innanzitutto, la separazione in quattro moduli con dipendenze unidirezionali garantisce che ogni livello conosca solo quello immediatamente sottostante; il domain non dipende da nient'altro, data dipende solo da domain, e così via.

Questo rende il codice più manutenibile e testabile: ad esempio, l'interfaccia **ReminderScheduler** vive nel modulo domain, mentre la sua implementazione concreta (**ReminderSchedulerImpl**, che usa **WorkManager**) sta in app. Questo permette di scrivere test unitari per **CalendarViewModel** fornendo un mock di **ReminderScheduler**, senza dover avviare **WorkManager** o Android stesso.

Il flusso è sempre UI -> ViewModel -> UseCase -> Repository (interfaccia) -> RepositoryImpl -> DB/API. Tutto lo stato della UI è contenuto in **CalendarUiState**, osservato tramite lo **StateFlow**: ogni modifica allo stato causa un recompose automatico solo dei composables che effettivamente dipendono da quel dato, evitando ricomposizioni inutili.

Il DB è alla versione 4, con migrazioni scritte esplicitamente. Quando è stata inserita **fallbackToDestructiveMigration**, tutti i miei eventi sono stati

eliminati, quindi le migrazioni esplicite evitano che gli utenti perdano i dati salvati quando l'app viene aggiornata.

**WorkManager** garantisce che le notifiche vengano recapitate anche se l'app è chiusa o il telefono viene riavviato. Ogni notifica è identificata da un tag univoco (basato sull'id dell'evento).

---

## Bacheche - Enrico Poli

Anche le bacheche si appoggiano alla stessa architettura a quattro moduli con dipendenze unidirezionali: non è stato necessario alcun compromesso architetturale per aggiungere la funzionalità, il flusso resta sempre in sequenza **UI, ViewModel, UseCase, Repository** (interfaccia), **RepositoryImpl DB/API**, e domain non conosce né Room né Retrofit.

La logica di filtro e riordino è testata da **BoardViewModelFilterTest**, che inietta implementazioni finte di **GetColumnsUseCase**, **GetCardsUseCase** e **GetBoardTagsUseCase** tramite **UseCasesProvider**: si può quindi verificare il comportamento dei filtri OR/AND senza avviare Room o un emulatore Android.

Le foreign key con **onDelete = CASCADE** tra bacheca/colonna, colonna/card e card/tag evitano dati orfani: eliminare una bacheca elimina automaticamente tutte le colonne, le card e i collegamenti ai tag al suo interno, senza doverlo gestire manualmente a livello di repository.

Il campo position viene ricalcolato (indicizzato con **mapIndexed**) ad ogni riordino di colonne o card, così l'ordinamento resta sempre una sequenza consistente senza “buchi”, anche dopo molti spostamenti.

Il drag&drop, sia delle colonne che delle card tra colonne diverse, è realizzato con la libreria reorderable, con un overlay personalizzato che segue il dito

durante il trascinamento e uno scroll automatico quando ci si avvicina ai bordi dello schermo: l'esperienza d'uso è quindi vicina a quella di un'app come Trello.

Infine, la scelta di cercare le immagini di copertina su Wikimedia Commons invece di richiedere un upload permette di evitare la gestione di file binari lato client e server: viene salvato solo l'url della foto, mentre il caricamento e la cache dell'immagine sono delegati alla libreria Coil.

.....

## **MIGLIORIE**

### **Aurora Loprino**

Personalmente, vorrei sincronizzare Chronio con Google Calendar per importare ed esportare eventi, e collegarlo con Chronio (Web) e Chronio (Java), così da avere un'esperienza coerente su più piattaforme con un singolo account. E' un progetto che ci ha accompagnato in vari esami, vederlo tutto unificato sarebbe una bella conclusione.

Questo richiederebbe però di sostituire (o affiancare) la persistenza locale di Room con un backend condiviso, e gestire la sincronizzazione e i conflitti tra dispositivi.

Un'altra miglione sarebbe il supporto ad eventi ricorrenti (es. "ogni lunedì", "ogni mese"), con la gestione delle eccezioni (es. saltare una singola occorrenza).

Si potrebbero anche aggiungere widget per la home screen e notifiche multiple per evento (es. un promemoria un giorno prima e uno un'ora prima).

Un'idea che mi piacerebbe esplorare è rendere le notifiche più "persistenti": se l'utente le elimina per errore senza aprirle, ricompaiono dopo un intervallo

configurabile (es. 10 minuti), così da ridurre il rischio di dimenticare un evento importante.

---

## **Enrico Poli**

Per quanto riguarda le bacheche, una miglioria che mi piacerebbe esplorare è la possibilità di scegliere se un tag debba essere universale (visibile e utilizzabile su tutte le bacheche) oppure locale a una singola bacheca, mostrandolo poi con colori o in liste differenti a seconda dell'ambito. Sulla stessa linea, si potrebbe collegare i tag delle bacheche con quelli già esistenti nel calendario, così da avere un'unica tassonomia condivisa tra le due funzionalità invece di due sistemi di tag separati.

Come già evidenziato da Aurora per il calendario, anche per le bacheche una miglioria naturale sarebbe rendere una bacheca condivisa tra più utenti, permettendo a più persone di vedere e modificare le stesse colonne e card. Anche qui servirebbe sostituire (o affiancare) la persistenza locale con un backend condiviso, oltre a gestire la sincronizzazione e gli eventuali conflitti tra dispositivi.

Un'altra miglioria sarebbe collegare una card della bacheca a un evento del calendario, così da assegnare una scadenza alle attività e rendere le due parti dell'app comunicanti tra loro, invece di lasciarle completamente separate come sono ora.

Infine, aggiungerei una searchbar nella BoardListScreen che permetta di cercare una card o una colonna per titolo o descrizione, così da trovare rapidamente un'attività anche quando le bacheche diventano numerose.